From raw images to reviewed cores

TMA QC Getting started.

Organize original images, reconstruct AF cores and register reviewed H&E pairs. Follow the same workflow from a terminal or Python.

Start with your folders

```
tma-study/
├── raw/                        Original NAS-style AF folders
├── he/HE_SLIDE_01/A1.tif        Original H&E core image
├── pairing/sections.csv        Reviewed section assignments
├── manifests/                  manifest.json → paired.json
├── geometry/                   Tile positions + held-out landmarks
├── models/                     Frozen illumination fields
├── configs/                    AF + research registration settings
└── runs/run01/                 acquisitions/ · pairs/ · reports/
```

This is a recommended dataset layout. The raw importer follows the hierarchy on pages 2–3; the manifest sets the locations of the other files.

What it does

Stitches raw AF tiles into TMA core images, applies illumination correction and reviews each stage. Optional registration aligns an explicitly assigned H&E core.

What you receive

Native TIFFs, coverage maps, source checksums and HTML reports that link each result to its original images.

01 Organize images 2–4

02 Prepare inputs 5–7

03 Run with Python 8–9

04 Review processing + QC 10–13

05 Register H&E cores 14–16

06 Choose methods + settings 17–18

07 Find code and help 19

Terms: TMA = tissue microarray; AF = autofluorescence; FOV = field of view; H&E = hematoxylin and eosin.

Edition 5.1 · 22 October 2026 · Package 0.4.1.
Examples use illustrative study labels. Scientific image quality requires review.

Organize your raw images

Point `--raw-root` at the folder containing the acquisition and slide folders. Keep each core and lamp setting distinct.

```
raw/                              ← --raw-root points here
└── acquisition01/
    └── SLIDE_01_AF/
        ├── A1/
        │   └── middle lamp 275/
        │       └── stitch/
        │           ├── FOV_1.tif
        │           ├── FOV_2.tif
        │           └── FOV_3.tif
        └── B2/
            └── middle lamp 275/
                └── stitch/
                    ├── FOV_1.tif
                    └── FOV_2.tif
```

Folder or file	Meaning in this tutorial
`acquisition01/`	Organizes one acquisition. Preserve original and repeat acquisition folders separately.
`SLIDE_01_AF/`	Source slide-folder label. The importer preserves this exact label in metadata.
`A1/`, `B2/`	TMA core positions. Labels such as A1 or B12 are recognized.
`middle lamp 275/`	The observed lamp-folder naming convention. The folder label supplies grouping information; its numeric suffix alone does not establish calibrated units or model compatibility.
`stitch/`	A candidate set of original AF tiles. In this raw hierarchy it contains tiles to stitch, not the final reconstructed core.
`FOV_1.tif`	One field of view. Numbered names such as `0001_0.6.099.tiff` are also recognized by their leading index.

Default image profile: one 2D uint16 AF plane per selected file, 2048 × 2048 pixels. Use an explicit compatible profile for a different tile size. The importer hashes files; image-format checks happen in `plan` and `run`.

Use your existing folders

The importer accepts project and `Raw` wrapper folders, including a repeated slide-folder name. Use the existing hierarchy when it follows these patterns.

Repeated slide folder

```
raw/project/Raw/SLIDE_01_AF/SLIDE_01_AF/
    A1/middle lamp 275/stitch/FOV_1.tif
```

Scan positions instead of a numbered stitch set

```
raw/acquisition01/SLIDE_01_AF/A1/middle lamp 275/
└── scan/
    ├── 43.59_42.35/100.tif
    └── 44.02_42.35/100.tif
```

Core folders with no lamp level

```
raw/SLIDE_01_AF/A1/stitch_folder/FOV_1.tif
raw/SLIDE_01_AF/A1/scan/43.59_42.35/100.tif
```

What the importer sees	What it does
Several valid candidate sets	Prefers `stitch`, then `stitch_folder`, then an unambiguous `scan` set.
Numbered tile files	Orders them numerically. The indices must be contiguous and start at 0 or 1.
One image per scan XY folder	Retains the XY-folder identity. Acquisition sequence remains unreviewed.
Several images per XY folder	Leaves the core blocked for explicit Z/exposure selection if no other valid candidate set exists.
Empty or unresolved core	Keeps it visible with `blocked_reasons`.
`stitch_folder_bg`**, autofocus or ZIP**	Counts the files in the inventory but does not select them as original raw FOVs.

Scan XY labels are source metadata. They are not automatically converted into native pixel positions. Geometry needs a reviewed coordinate source or the explicitly configured geometry backend.

The folder importer is for raw AF acquisitions. H&E inputs enter later through explicit section-pair assignments.

Choose where files live

The code checkout, original images, processing assets and generated results serve different roles.

```
code/phomedics-tma-qc/        Installed source checkout

/data/tma-study/
├── raw/                      Original AF folders from pages 2–3
├── he/HE_SLIDE_01/A1.tif      Original H&E core crop
├── pairing/sections.csv
├── manifests/
│   ├── manifest.json         Reviewed AF inventory
│   └── paired.json           AF + explicit H&E assignments
├── geometry/                 Native positions + landmarks
├── models/group01/           flatfield.tif · darkfield.tif
│                             model.json
├── configs/af.toml
├── configs/core-registration.toml
└── runs/run01/

/local-ssd/tma-qc-cache/      Writable runtime cache
```

Path in the manifest	How the code resolves it
`raw_root`	Use the absolute raw-input directory, such as `/data/tma-study/raw`.
Relative FOV `path`	Under `raw_root`, for example `acquisition01/SLIDE_01_AF/A1/…/FOV_1.tif`.
Relative model or geometry path	Beside the manifest. From `manifests/manifest.json`, use `../geometry/A1.json` and `../models/group01/flatfield.tif`.
Pairing CSV asset path	Relative to `pairing/sections.csv`, such as `../he/HE_SLIDE_01/A1.tif`. `import-pairs` stores absolute asset paths in its output manifest.
Absolute asset path	Used directly. Update the path when moving to another machine.

Writable locations: the manifest output, results and cache must be outside the raw archive. Set `runtime.cache_root` to local SSD. If omitted, the software default is the system temporary directory's `tma-qc` folder; the code does not automatically choose an SSD.

Our copied NAS root:
`/mnt/pho-nas/home/data/TMA_raw/2026`
The original acquisition root is `/mnt/pho-nas/datacollection/TMA(Raw)/2026`. Use a stable, complete selected input set.

Understand the manifest

Users supply a raw root. The importer lists the candidate tiles and records their identities and checksums in `manifest.json`.

Field	Role
`dataset_id`	The study/collection identifier.
`cores[].slide`	The assigned slide identifier. Automatically imported values use cleaned folder labels and hash suffixes.
`core`, `acquisition`, `channel`	The core position, acquisition record and AF channel assignment.
`cores[].uid`	The unique core-acquisition reference used by processing and H&E pairing.
`group`	A reference to an illumination-group/model assignment.
`fovs[]`	Selected tile IDs, original paths and full source SHA256s.
`positions`	Path to native tile geometry when using the default supplied-geometry backend.
`pairs[]`	Optional AF–H&E section-pair assignments.

Keep the original image identity

```
"metadata": {
  "source_slide_label": "SLIDE_01_AF",
  "source_lamp_label": "middle lamp 275",
  "source_folder": "acquisition01/SLIDE_01_AF/A1/middle lamp 275",
  "source_FOV_set_verified": false
}
```

This is an illustrative fragment, not a complete manifest. Use the generated JSON rather than recreating machine IDs by hand.

Name hash

The short suffix on an imported ID distinguishes source labels. IDs depend on relative acquisition and lamp paths.

File checksum

The full SHA256 binds the record to the image bytes. It is a separate value from the short ID suffix.

Folder labels need their acquisition meaning. The importer preserves the original spelling; it does not establish a laboratory barcode, physical section identity or an AF–H&E match from that spelling alone.

Create your manifest

In an activated Python 3.11+ environment, install from the repository checkout. Run these commands in a terminal:

```
python -m pip install -e '.[codecs]'
python -m tma_qc --help
```

Run the Python module against the raw root from page 2. Write the manifest outside it:

```
python -m tma_qc import-folder \
  --raw-root /data/tma-study/raw \
  --dataset-id TMA-STUDY-01 \
  --out /data/tma-study/manifests/manifest.json
```

The importer hashes selected tiles, preserves source labels and reports candidate and blocked cores. It works directly from the folders.

Check what was found

```
python - <<'PY'
import json
from pathlib import Path
path = Path("/data/tma-study/manifests/manifest.json")
d = json.loads(path.read_text())
for c in d["cores"]:
    print(c["uid"], c["core"], len(c["fovs"]),
          c["blocked_reasons"])
print("Model groups:", [g["id"] for g in d["groups"]])
PY
```

Generated automatically	Still needs acquisition information
Candidate slide/core/acquisition/lamp records	Correct identity, channel and acquisition completeness
Tile paths, IDs and source hashes	Native geometry and any ambiguous plane selection
Draft illumination groups	Compatible frozen model assignments and optional H&E pairs

A draft is a successful import. Exit 0 means the JSON was generated; it may contain blocked cores. Use a new output filename to import again. Use `plan` or `run` to check processing prerequisites.

Hashing reads the selected files and can take time on NAS. Read errors, symbolic links or detected source changes stop publication. Use `import-copy` to reuse a completed error-free transfer inventory.

Add tile positions and models

The default workflow uses supplied native tile positions and frozen CIDRE illumination fields. Folder discovery prepares the image list; these assets make reconstruction executable.

1. Add a position for each tile

Save under `geometry/` and set the core's `positions` path in the manifest. Match every FOV ID exactly once.

```
{
  "coordinate_space": "native_pixels",
  "axes": "XY",
  "origin": "upper_left",
  "positions": [
    {"id": "1", "x": 0, "y": 0},
    {"id": "2", "x": 1600, "y": 0}
  ]
}
```

The positions above are illustrative, not a calibration for your scanner. Current blending requires nonnegative integer native-pixel coordinates. Set `geometry_reviewed` only after review. Optional MIST estimation requires explicit grid/sequence information and a working Java/classpath setup.

Stage 4 checks and flags for review. Geometry QC records overlap measurements and highlights the largest measured shifts in preview crops. It leaves tile positions and FOV order unchanged. Unavailable overlap measurements remain unassessed.

2. Assign a compatible correction model

```
"groups": [{
  "id": "COPY_THE_GENERATED_GROUP_ID",
  "flatfield": "../models/group01/flatfield.tif",
  "darkfield": "../models/group01/darkfield.tif",
  "model_metadata": "../models/group01/model.json"
}]
```

This is a manifest fragment. Keep the generated group ID used by its cores and replace the asset paths with real reviewed files. Compatibility depends on the acquisition settings. Folder similarity alone is insufficient.

The flatfield and darkfield are frozen correction fields. The model receipt records their origin and expected checksums. To fit new fields, use the separate `fit-cidre` workflow with a compatible pool and the pinned Java runtime.

Scientific review remains separate. Successful field application does not establish absolute fluorescence preservation. See `docs/METHODS.md` and `docs/VALIDATION.md`.

Process your first core

Copy `configs/af2048.toml` from the checkout to your dataset's `configs/af.toml`. Set the local cache location. The normal defaults use CIDRE correction and supplied geometry.

```
[runtime]
cache_root = "/local-ssd/tma-qc-cache"
threads = 2
tile_workers = 4

[profile]
shape = [2048, 2048]
dtype = "uint16"
```

Use the actual core UID printed on page 6 in place of `CORE_UID_FROM_MANIFEST`. An optional read-only readiness check is:

```
python -m tma_qc plan \
  --manifest /data/tma-study/manifests/manifest.json \
  --config /data/tma-study/configs/af.toml \
  --out /data/tma-study/runs/run01 \
  --core CORE_UID_FROM_MANIFEST
```

When the selected core's prerequisites are ready, process it:

```
python -m tma_qc run \
  --manifest /data/tma-study/manifests/manifest.json \
  --config /data/tma-study/configs/af.toml \
  --out /data/tma-study/runs/run01 \
  --core CORE_UID_FROM_MANIFEST \
  --resume
```

Repeat `--core` to select more cores. Omit it to process all manifest cores. Rerun the same command with `--resume` to reuse compatible verified outputs; code or input changes can require rebuilding stages.

Try the software first

`python -m tma_qc demo --out /tmp/tma-qc-demo`
Use a new directory. This generates runnable synthetic inputs and a small 160 × 160 tile profile.

Read the result

Open `runs/run01/reports/index.html` in a browser. For `run`: exit 0 = execution complete, 1 = error, 2 = blocked.

Use TMA QC from Python

Prepare manifests and run the same recorded stages from scripts and notebooks. Method selectors and settings are shared with the terminal interface.

`phomedics-tma-qc` is the installed package; `tma_qc` is the Python module. `tma-qc` is the terminal shortcut for `python -m tma_qc`.

Discover original AF images

```
from tma_qc.api import import_folder, import_pairs, run

draft = import_folder(
    raw_root="/data/tma-study/raw",
    output="/data/tma-study/manifests/manifest.json",
    dataset_id="TMA-STUDY-01",
)
print("Core acquisitions:", len(draft["cores"]))
```

Use a new output path. Review geometry, model assets and acquisition identity before running. For H&E pairs, prepare the CSV on page 14 and call:

```
paired = import_pairs(
    manifest="/data/tma-study/manifests/manifest.json",
    pairs_csv="/data/tma-study/pairing/sections.csv",
    output="/data/tma-study/manifests/paired.json",
)
```

Run a prepared manifest

```
summary = run(
    manifest="/data/tma-study/manifests/paired.json",
    output="/data/tma-study/runs/run01",
    config="/data/tma-study/configs/af.toml",
    resume=True,
)
print(summary["execution_errors"], summary["blocked_stages"])
```

Use `manifest.json` for AF-only processing. Optional `selected=["CORE_UID"]` limits processing to actual core UIDs and their pairs. The result is a summary dictionary; invalid inputs may raise exceptions.

One workflow. Both interfaces save the same source checks, outputs, stage receipts and offline reports. Earlier `import_raw_folder` and `run_pipeline` imports remain available.

Connect each process to QC

Processing produces an image or a coordinate record. Its QC reads those results and saves measurements, evidence and review status separately.

Stage	What to review
1 · `index`	Hashes, single raw plane, stored pixel segments and expected profile. Verifies listed files; missing physical acquisitions need an expected inventory.
2 · `tile_qc`	Local detail, saturation and zeros in raw FOVs. Unavailable-window reasons and native sampling coverage accompany the measurements.
3 · `geometry`	Native tile positions: supplied by default, optionally estimated with reviewed MIST inputs.
4 · `geometry_qc`	Similarity and residual shifts in expected overlaps. Advisory flags leave FOV order and positions unchanged.
5 · `illumination`	Corrected tiles from the frozen flatfield and darkfield.
6 · `illumination_qc`	Raw/selected overlap agreement, assessed coverage, signal spans and model gains. See page 11.
7 · `mosaic`	Native raw and selected mosaics built with the same geometry and blending.
8 · `mosaic_qc`	Localized seam tails, global/local brightness patterns and acquisition coverage. See page 12.
9 · `tissue`	Provisional possible-tissue and unknown covered-area maps.

Read execution and quality together

Execution

Did the stage complete, fail or wait for a prerequisite? A diagnostic crash is a software execution error.

Quality

`review` means inspect available evidence. `not_assessed` means a measurement is unavailable; it is not a pass.

Keep the reconstruction available. Diagnostic failure does not suppress an otherwise valid stitched image. Essential missing inputs can block dependent processing. Scientific acceptance requires review.

Focus is advisory. Constant or weak-detail windows cannot establish blur. Window/interior coverage is geometric sampling, including unavailable windows; it is not a blurred-tissue percentage. Tile-grid previews average counts and use native-histogram display limits.

Review illumination correction

Does the same tissue agree better across overlapping tiles, and what happened to its signal range?

01	02	03	04
Raw tiles + geometry	Frozen correction fields	Same supported pixels	Before/after evidence

Evidence	How to interpret it
Assessed / candidate pairs	How many geometric overlaps supplied measurable comparison evidence. This fraction describes pairs, not tissue area.
Exclusion reasons	Insufficient raw support, unavailable raw correlation or correlation below the selection rule. Excluded pairs remain unknown.
Mismatch measurements	Relative mean difference and matched-pixel differences, including p95 and worst-pair crops. A mean can hide equal and opposite local errors.
Signal spans + gains	Per-FOV raw/selected percentiles and span ratios at identical sampled coordinates, plus model-gain percentiles.
Exact signal collapse	Native extrema show that varying raw counts became constant. Inspect the source, model and selected image for the cause.

Start with the evidence coverage

A pair is eligible with at least **256 raw-supported samples** and **raw correlation ≥ 0.8**. These are uncalibrated selection rules, not image-quality acceptance thresholds. The same raw-selected pairs and pixels are used after correction.

Then inspect agreement and signal together

Lower mismatch supports improved overlap agreement. Signal compression can also reduce absolute differences. Relative mismatch avoids a fixed one-count denominator floor; two zero means are unassessed. Span ratios and model gains remain review measurements without automatic cutoffs.

> **Agreement is one part of validation.** It does not establish absolute fluorescence preservation, independent geometric accuracy or performance across excluded overlaps.

Open the core report's illumination section, overlap crops and `illumination-qc.json`. Retain the raw reference and compatible model receipt.

Review the assembled mosaic

Check native seam evidence, repeating brightness patterns and acquired support. Read local tails alongside whole-core summaries.

1. Find localized seam changes

Known overlap midlines are measured across adjacent native strips, with nearby control cuts where possible. The report retains seam medians, **p95 and maximum** steps. Distinct-seam counts include measurable low-signal and mixed strata; stratum counts remain separate.

Worst-seam records identify the FOV pair, native cut and largest measured line. A localized brightness step can remain visible even when the median is zero. Isolated peaks may be noise or anatomy; inspect p95, controls and the images together.

2. Compare global and local tile-pitch patterns

The fit uses every supported native row or column bin. Four orthogonal strips per axis preserve regional evidence that can cancel in a global average. Both before/after fits use the same raw-defined pixel support.

Saved measurement	Meaning
Harmonic amplitudes	Count amplitudes at the first three fitted harmonics of tile pitch.
Explained fraction	How much detrended projection variation the fitted harmonics explain. It is not an artifact probability.
Support + region	Native sample counts, strip coordinates and unavailable reasons. Global and local fit counts are shown separately.

Accumulation uses bounded 512 × 512 source patches. Preview profiles alone are averaged; their reduction cannot alias the native fit. Missing native bins stay excluded, and display bins containing a gap remain unknown.

3. Inspect coverage and original tissue

Use the shared-window raw/selected comparison, native TIFFs and `coverage.tif`. Coverage marks acquired image support. It does not prove the entire intended core was captured.

Keep the result advisory. Anatomy can repeat at tile spacing. Patterns may cancel within one strip or occur at other periods. A low score cannot certify a clean core, and no artifact-area fraction is reported.

Review your stitched images

Each AF core has nine recorded stages: `index`, `tile_qc`, `geometry`, `geometry_qc`, `illumination`, `illumination_qc`, `mosaic`, `mosaic_qc` and `tissue`.

```
runs/run01/
├── reports/index.html          Start reviewing here
├── reports/core-qc.csv         Per-core summary
├── stage-results.json          Stage outputs and checksums
└── acquisitions/
    └── <acquisition>/<core>/AF/<core_uid>/
        └── mosaic/<stage-key>/
            ├── raw-mosaic.tif
            ├── mosaic.tif
            ├── coverage.tif
            └── stage.json
```

Output	Meaning
`raw-mosaic.tif`	Native stitched reference built from original raw tiles.
`mosaic.tif`	The selected stitched image. CIDRE-corrected under the default illumination setting.
`coverage.tif`	Geometric acquisition coverage, showing where tiles contribute data.
QC pages and evidence	Focus/detail diagnostics, overlap checks, seam/brightness measurements and provisional tissue support for review.

Both mosaics retain the full acquisition canvas at native resolution as float32 BigTIFFs. Complete core coverage depends on a complete intended tile set and appropriate geometry.

Current filenames are product names. Slide/core identity is carried by the UID folders, manifest and stage records. Retain these records when moving images. A filename scheme embedding original slide IDs remains a proposed export improvement.

The separate source-first publication recipe exports the selected image as `corrected.tif` inside `results.zip`, together with manifests and QC records. It requires an independently verified complete AF core.

Execution and image quality are separate. A completed stitch is available for inspection even when a diagnostic stage fails. Current scientific image-quality acceptance requires review.

Pair AF and H&E cores

Reuse your AF manifest. Add already cropped H&E cores and one pairing CSV; the importer generates the paired manifest.

```
tma-study/
├── he/HE_SLIDE_01/A1.tif
├── pairing/sections.csv
└── manifests/
    ├── manifest.json
    └── paired.json
```

Each H&E file is one self-contained core-crop TIFF plane, grayscale or RGB; native pyramids are supported. Prepare whole-slide crops separately. Keep AF acquisition folders in place.

Find the AF core UID

From `tma-study/`, list UID, source folder/acquisition and core label. Choose the acquisition you reviewed.

```
python - <<'PY'
from tma_qc.io.manifest import Manifest
for c in Manifest("manifests/manifest.json").cores:
    print(c.uid, c.metadata.get("source_folder", c.acquisition), c.core)
PY
```

Write one reviewed pair per CSV row

Save `pairing/sections.csv`. Choose your own pair `uid`; copy the exact AF UID into `af_core`. The H&E path resolves beside this CSV. Confirm the section relationship from your records.

```
uid,af_core,he_path,relationship
PAIR_01,CORE_UID_FROM_MANIFEST,../he/HE_SLIDE_01/A1.tif,neighboring_sections
```

Generate the paired manifest

```
python -m tma_qc import-pairs \
  --manifest /data/tma-study/manifests/manifest.json \
  --pairs /data/tma-study/pairing/sections.csv \
  --out /data/tma-study/manifests/paired.json
```

Hashes and the paired JSON are generated for you. Use a new output filename. Original AF and H&E slide IDs can differ; a shared core label alone does not establish a pair.

Run your first registration

Start with the built-in global geometry/SIFT fit and review its result. Advanced assets and method tuning can follow.

Enable the fitter in your AF configuration

Edit the existing `[registration]` section in `configs/af.toml`, or add it if absent. Keep your AF and cache settings.

```
# configs/af.toml — alongside AF and runtime settings
[registration]
backend = "core"
core_method = "geometry_sift"
```

Built-in defaults export aligned H&E. This first run needs no second TOML, supplied transform, landmark file or optional model package.

Run one AF core and its assigned H&E pairs

```
python -m tma_qc run \
  --manifest /data/tma-study/manifests/paired.json \
  --config /data/tma-study/configs/af.toml \
  --out /data/tma-study/runs/run01 \
  --core CORE_UID_FROM_MANIFEST --resume
```

Use the AF UID from your CSV. Compatible AF stages are reused on resume; source hashes are checked again. A pair adds `he_qc`, `registration` and `registration_qc`.

Open the report and aligned image

```
runs/run01/reports/index.html
runs/run01/pairs/<pair_uid>/registration/<stage-key>/
    aligned-he.tif   transform.json   overlay.png
```

The AF mosaic is the fixed reference. H&E is sampled onto its native canvas. Review original sections, the overlay and geometric overlap. Without landmarks, independent landmark accuracy is reported unavailable.

Later options. Page 16 shows independent landmarks. `examples/section-pairs.advanced.example.csv` adds supplied transforms and calibrated pixel sizes. Method tuning, RoMa and local refinement are covered in `docs/RESEARCH_GUIDE.md`.

Review alignment independently

Pair stages are `he_qc`, `registration` and `registration_qc`. Open the pair report, original sections and native exports together.

```
runs/run01/pairs/<pair_uid>/registration/<stage-key>/
├── transform.json
├── aligned-he.tif                When export is enabled
├── geometric-overlap.tif
├── overlay.png
├── candidates.json              Core fitting evidence
├── fit-correspondences.csv
├── refinement.json
└── local-pull-displacement.npy   Only when a local warp is used
```

The original H&E image is sampled onto the native AF canvas once, in bounded chunks. `transform.json` declares H&E → AF direction and native pixel coordinates. Retain its companion residual field and coordinate maps when present; the global affine alone cannot reproduce a local warp.

Optional: supply held-out landmarks

Use declared `evaluation_only` / `native_pixels` JSON, or a CSV with these columns. The two rows below illustrate the format; the default provisional tolerance check requires at least five landmarks.

```
he_x,he_y,af_x,af_y
120.0,180.0,128.0,175.0
420.0,310.0,428.0,305.0
```

Set `landmarks` in the pairing CSV to `../geometry/PAIR_01-landmarks.csv`. Choose shared structures across the core independently from fitting. Coordinates must be distinct on both images; optional IDs must be nonempty and unique. Uniqueness alone does not establish biological independence. These points are never passed to the fitter.

After editing the CSV, re-import from the original AF manifest into a new file such as `paired-evaluation.json`, then run that manifest. CSV edits do not update an earlier paired JSON.

Evidence	What it establishes
Fit matches and residuals	Internal fitting support, not independent alignment accuracy.
Held-out landmark errors	Local errors at sampled shared structures, reported in native AF pixels, including the local field when present.
Geometric overlap	Acquired geometric support, not dense correspondence or a usable-tissue mask.

All results require review. Neighboring sections differ. Sparse landmarks, model confidence and positive transform Jacobians cannot establish dense anatomical accuracy or restore physically folded tissue.

Choose a method

Start with the defaults. Select another supported method when your study needs it, and keep the same manifests, stage sequence and QC records.

```
python -m tma_qc methods
python -m tma_qc run --help
```

Choice	Available methods
Geometry	`provided` (default) or `mist`. MIST needs Java and a reviewed tile grid.
Illumination	`cidre` (default) applies frozen fields; `none` retains original counts.
Registration	`provided` (default), experimental `sift_affine`, or integrated `sift`, `orb`, `geometry_sift`, `geometry_roma`.
Core transform	`similarity` (default) or `affine`. The latter also allows shear and unequal scaling.
Local refinement	`none` (default) or optional `bspline_mi`.

Change a method at the terminal

```
python -m tma_qc run \
  --manifest /data/tma-study/manifests/paired.json \
  --config /data/tma-study/configs/af.toml \
  --out /data/tma-study/runs/geometry-sift \
  --registration-method geometry_sift \
  --registration-transform similarity \
  --set tile_qc.stride=128 --resume
```

`plan` accepts the same method and parameter options. For comparisons, keep inputs and evaluation points fixed, choose a separate run directory, and change one method or setting at a time.

Asset assignments take precedence. Supplied tile positions are used before MIST; a supplied pair transform is used before fitting. RoMa needs its extra and cached weights; B-spline/MI needs its extra. Missing prerequisites block that path.

Tile detail QC, mosaic blending and tissue support currently each use one method with tunable parameters. Geometry QC flags evidence for review and never changes tile order.

Tune and reproduce settings

Keep common settings and detailed core matching in one TOML. Omitted values retain their defaults.

```
[registration]
backend = "core"
core_method = "geometry_sift"

[registration.matching]
max_dimension = 1024
transform = "similarity"
min_inliers = 12

[registration.refinement]
backend = "none"
```

Merge these tables into your existing main file. Existing external `core_config` files still work; inline keys override only the corresponding research settings.

Use the same choices in Python

```
from tma_qc.api import run, settings
effective = settings(registration_method="geometry_sift")
summary = run(
    manifest="/data/tma-study/manifests/paired.json",
    output="/data/tma-study/runs/geometry-sift",
    config="/data/tma-study/configs/af.toml",
    registration_method="geometry_sift",
    parameters={"tile_qc": {"stride": 128}}, resume=True,
)
```

One precedence rule

Defaults → TOML → parameter overrides → explicit method selectors. CLI overrides use repeated `--set SECTION.KEY=VALUE` with typed TOML values; Python uses nested dictionaries. Unknown names and invalid values are errors.

Repeat your intended settings when resuming. `run --resume` uses that invocation's arguments. Effective main settings are saved in `run.json`; pair receipts also save resolved core settings. Relevant changes rebuild affected stages.

`docs/SETTINGS.md` lists all defaults, units and backend applicability.
`configs/methods.example.toml` is an annotated starting point.
Core methods use `registration.matching.max_dimension`; main `registration.max_dimension` belongs to legacy `sift_affine`.

Find code and get help

The CLI runs the workflow; offline HTML reports show the results. The source is organized by responsibility.

Source location	Responsibility
`api.py` / `cli.py` / `settings.py`	Public interfaces, method choices and shared setting resolution.
`io/` / `schema.py`	Folder/manifest input, source identities and file contracts.
`workflows.py` / `context.py`	The stage sequence and per-core/per-pair inputs.
`pipeline.py`	Dependencies, verified resume, checkpoints and stage records.
`stages/`, `backends/`, `qc/`	Processing steps, numerical implementations and diagnostics.
`reporting/` / `tests/`	Core/batch review pages and software/scientific controls.

Common first-run questions

Symptom	Check next
No recognized core folders	Point the raw root above the slide/core hierarchy. Compare pages 2–3.
Blocked FOV set	Inspect `blocked_reasons`: missing indices, empty cores or ambiguous scan planes.
Missing field or positions	Attach compatible model assets and complete native geometry; check relative paths.
Source changed / hash mismatch	Check acquisition/copy activity and the expected source. Import a new reviewed snapshot rather than editing hashes to bypass verification.
Registration blocked	Verify AF UID, H&E checksum, selected method, optional packages and cached weights.

Continue in the repository

`examples/README.md` — supported input layouts and templates.
`docs/RESEARCH_GUIDE.md` — preparation, run and review workflow.
`docs/SETTINGS.md` — method choices, parameter defaults and units.
`docs/pipeline.md` — file contracts, coordinates and output layout.
`docs/METHODS.md`, `docs/STAGE_QC.md` — calculations and QC limits.
`docs/DEVELOPMENT.md` — extending code and running `make check`.

Reference: mahmoodlab/TRIDENT and TRIDENT documentation, reviewed 20 October 2026. This guide is based on the local PhoMedics source and examples; TRIDENT is an onboarding reference, not a dependency.

The tutorial's folder examples are illustrative. Existing scientific data, transfer state and frozen results are outside the tutorial build. The PDF can be regenerated from `docs/tutorial/user-guide.html`.